

mytext2sql

Arquitetura de um text to SQL em português, com modelo local no Ollama, validação por AST e banco somente leitura.

Visão geral

A ideia central do projeto é que **o modelo nunca é confiável**. Ele é tratado como um gerador de rascunho que pode errar ou ser manipulado, e tudo em volta dele existe pra conter isso. O fluxo completo é:

```
Pergunta (PT)
→ prompts.py monta as mensagens (system + schema + regras + 10 exemplos)
→ llm.py chama o Ollama (temperature 0, saída JSON forçada por schema)
→ sql_guard.py valida e reescreve o SQL (AST via sqlglot)
→ db.py executa com usuário read only, timeout e teto de linhas
→ ui.py mostra tabela + gráfico automático

pipeline.py orquestra tudo e faz retry (até 3x), devolvendo o erro pro modelo
```

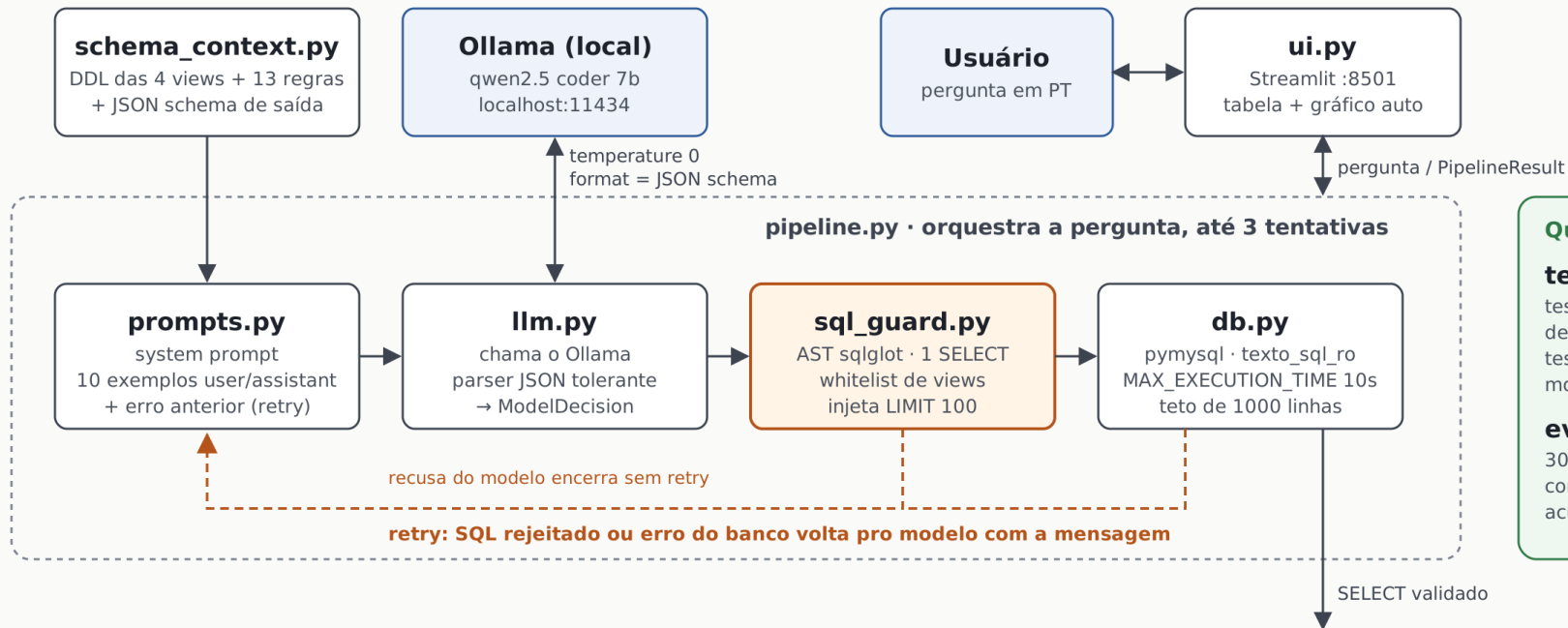
O diagrama completo está na próxima página.

Camadas de defesa, resumidas

Camada	Onde	O que impede
1 · Contexto restrito	schema_context.py	Modelo nem sabe que as tabelas tb* existem
2 · Validação por AST	sql_guard.py	Comandos de escrita, múltiplos statements, tabelas fora da whitelist, funções perigosas, comentários versionados
3 · Permissão no banco	x91_usuario_readonly.sql	Qualquer coisa que passe pelo guard e não seja SELECT nas 4 views
4 · Limites de recurso	db.py	Queries longas e resultados enormes

mytext2sql · arquitetura

Pergunta em português → SQL gerado por modelo local → validado → executado só leitura → resultado



Qualidade

tests/

test_sql_guard.py: ~25 casos de ataque e regras de LIMIT
test_pipeline.py: retry com modelo e banco mockados

eval/

30 perguntas com gabarito compara RESULTADO, não texto acurácia por nível, tag, modelo

Camadas de defesa

1 · Contexto restrito

modelo só conhece as views

2 · sql_guard (AST)

1 SELECT, whitelist, sem /*! reserializa sem comentários

3 · Permissão no banco

GRANT SELECT só nas views

4 · Limites de recurso

timeout no servidor
teto de linhas

1. Camada de dados (mysql/)

Três scripts rodam em ordem alfabética no docker-entrypoint-initdb.d. O prefixo x9 foi escolhido pra garantir que rodem depois do dump.sql.

dump.sql

Cria as tabelas cruas (tborders, tborderdetail, tbcustomers, tbproducts, tbcategories) e carrega os dados.

x90_views.sql: a camada semântica

É o coração do projeto. Quatro views com nomes e colunas em português:

- **vw_vendas:** grão de item de pedido, já desnormalizada (cliente, produto, categoria, ano, mês, ano_mes). Parte de tborderdetail com LEFT JOIN porque as FKs são ON DELETE SET NULL: uma linha órfã não pode sumir do faturamento. Extrai cidade e UF do campo Address e calcula preco_unitario_praticado.
- **vw_clientes** e **vw_produtos:** dimensões simples.
- **vw_referencia:** uma única linha com MAX(orderdate). Resolve um problema clássico: os dados de exemplo não chegam até hoje, então "último mês" com CURDATE() daria vazio. O modelo é instruído a sempre fazer CROSS JOIN vw_referencia pra períodos relativos.

O ganho das views é duplo: o modelo erra menos (colunas autoexplicativas, sem joins pra montar) e a superfície de ataque encolhe.

x91_usuario_readonly.sql

Cria o texto_sql_ro, faz REVOKE ALL primeiro (idempotência) e dá GRANT SELECT só nas 4 views. Esse usuário não enxerga as tabelas tb* de jeito nenhum.

2. Contexto do modelo (schema_context.py + prompts.py)

schema_context.py é a **fonte única das regras de negócio**, com três blocos:

- **VIEWS_DDL:** um DDL "fake" das views, com comentário em cada coluna. É isso que o modelo lê, não o schema real.
- **REGRAS:** 13 regras explícitas que codificam armadilhas reais dos dados. Faturamento é SUM(valor_total_item) e nunca multiplica por quantidade (já vem multiplicado); pedidos é COUNT(DISTINCT pedido_id); ticket médio tem fórmula fixa; mapeamento de categorias PT para EN (livros → 'Digital Books', notebooks nunca vira "cadernos"); estados por extenso viram sigla; recusa obrigatória pra qualquer pedido de alteração.
- **JSON_SCHEMA:** o contrato de saída {pode_responder, sql, explicacao, motivo}.

prompts.py monta a conversa. Os 10 exemplos few shot entram como **pares user/assistant reais** na lista de mensagens, não como texto colado no system prompt: o modelo "vê" a si mesmo respondendo no formato certo. Um dos exemplos é propositalmente uma recusa ("Apague todos os pedidos do cliente Joao Silva"), ensinando o caminho do pode_responder=false. Todo SQL dos exemplos foi executado contra o banco antes de entrar ali.

Na retentativa, build_messages anexa à pergunta o erro anterior e o SQL que falhou, pedindo correção.

3. Cliente do LLM (llm.py)

- Chama ollama.Client.chat com format=JSON_SCHEMA (saída estruturada nativa do Ollama), temperature 0 pra ser determinístico e num_ctx 8192 (o contexto fixo mede uns 2.1k a 2.5k tokens, então sobra folga).

- Parser tolerante em cascata: tenta `json.loads` direto, depois bloco markdown json, depois o maior `{...}` do texto. Se nada der certo, devolve uma recusa em vez de quebrar.
 - `_extract_sql` limpa o caso em que o modelo enfia um bloco sql dentro do próprio campo JSON.
- O resultado é um `ModelDecision`, desacoplado do formato bruto.

4. Guardiã do SQL (`sql_guard.py`)

Primeira camada de segurança, sólida porque trabalha na **AST**, não em regex sobre texto:

1. Rejeita comentário versionado do MySQL (`/*! ... */`). O `sqlglot` trata isso como comentário inerte, mas o servidor MySQL **executa** o conteúdo. Sem essa checagem, a AST pareceria segura escondendo um comando real.
2. Faz parse com dialeto MySQL e exige **exatamente um** statement.
3. A raiz tem que ser `Select` ou `Union` (então `WITH ... SELECT` passa).
4. Bloqueia `SELECT ... INTO` (cobre `INTO OUTFILE`).
5. Whitelist de tabelas: toda `exp.Table` tem que estar nas 4 views, exceto nomes de CTEs definidos na própria query.
6. Blacklist de funções perigosas: `SLEEP`, `BENCHMARK`, `LOAD_FILE`, `GET_LOCK` e afins.
7. Injeta `LIMIT 100` se não houver, exceto em agregação de linha única (sem `GROUP BY` e só agregados ou literais na projeção).

O SQL devolvido é sempre **reserializado a partir da AST** com `comments=False`, nunca a string original. Isso por si só descarta qualquer truque escondido no texto bruto.

5. Execução (`db.py` + `config.py`)

`config.py` é o único lugar que lê `os.environ`, com `dataclass` imutável (`frozen=True`). `db.py` conecta como `texto_sql_ro` e tem três proteções de recursos:

- `SET SESSION MAX_EXECUTION_TIME` mata a query **no servidor** se passar do tempo (10s por padrão).
- `read_timeout` do lado do cliente um pouco acima disso, como rede de segurança.
- `fetchmany(max_rows + 1)`: busca uma linha a mais que o teto (1000) só pra saber se truncou, sem puxar o resultado inteiro pra memória.

6. Orquestração (`pipeline.py`)

O loop de até 3 tentativas tem três saídas possíveis:

- **Modelo recusa**: retorna na hora, sem `retry` (recusa é decisão, não erro).
- **Guard rejeita ou banco dá erro**: registra, monta o `erro_anterior` com a mensagem e o SQL, e tenta de novo. É o mecanismo de autocorreção: o modelo recebe feedback concreto.
- **Sucesso**: devolve `PipelineResult` com SQL final, explicação, `DataFrame`, tentativas, tempo e erros intermediários.

7. Interface (`ui.py`)

Streamlit simples: pergunta, explicação em negrito, SQL num expander, tabela e um gráfico automático por heurística (coluna temporal como ano, mes, ano_mes vira linha; categórica com até 50 linhas vira barra). A sidebar mostra o modelo em uso e o histórico da sessão. O `sys.path.insert` no topo existe porque o Streamlit roda o arquivo como script solto, não como pacote.

8. Qualidade (`tests/` + `eval/`)

Duas coisas diferentes e complementares:

- **tests/**: unitários determinísticos. `test_sql_guard.py` cobre cerca de 25 casos (DROP, INSERT, information_schema, SLEEP, INTO OUTFILE, comentário versionado, regras de LIMIT). `test_pipeline.py` usa monkeypatch no modelo e no banco pra testar o retry sem Ollama nem MySQL.
- **eval/**: 30 perguntas com gabarito, nível e tags. Compara o **resultado da execução**, não o texto do SQL (dois SQLs diferentes podem estar ambos certos), ignorando ordem quando o gabarito não tem ORDER BY e com tolerância de 0.01 em decimais. O `--model` permite comparar modelos com relatório por nível e por tag.

Pontos de atenção

1. **Senha do texto_sql_ro commitada num repo público** (`x91_usuario_readonly.sql` e `.env.example`). O estrago é limitado porque o usuário só lê as views, mas se o banco algum dia ficar exposto vale trocar e tirar do repo.
2. **README diz que a imagem é mysql:8.0**, mas o `docker-compose.yml` usa `dbconsultoria/mysql-mysql:1.0`. Vale alinhar.
3. **Compose com caminhos absolutos** `D:/Github/...` e dependência da rede externa `lakehouse_net`. Quem clonar em outra máquina vai quebrar nos dois. Caminhos relativos e uma nota sobre criar a rede resolvem.
4. **O guard não valida o schema qualificado** (`outrobanco.vw_vendas` passaria). Na prática o GRANT segura, que é justamente o motivo das duas camadas, mas dá pra fechar no guard checando `table.db`.